<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN"> <html><head> <title>404 Not Found</title> </head><body> <h1>Not Found</h1> The requested URL was not found on this server. </body></html>

3

RL Fundamentals: The Complete Picture

What Is Reinforcement Learning?

Reinforcement Learning (RL) is learning through trial and error with feedback. It is how you learned to ride a bike:

1. Try something (turn the handlebars to the left)
2. See what happens (you start to fall)
3. Get feedback (negative, that was bad!)
4. Adjust your strategy (the next time, turn less sharply)
5. Repeat until you learn what works

For AI language models, the process is similar:

1. AI generates a response
2. Humans evaluate it (good/bad/meh)
3. AI learns to generate more responses like the good ones
4. Repeat millions of times
5. AI becomes helpful!

Supervised Learning = Learning from a textbook with all answers shown. The teacher says "The capital of France is Paris," you memorize it, and repeat it back. Good for facts, patterns, and specific examples.

Reinforcement Learning = Learning from real-world trial and error. The teacher says "Learn to cook a delicious pasta." You try adding lots of salt (tastes bad), then a little salt (tastes okay), then salt and herbs (tastes great). The teacher tells you which was best. Good for skills, judgment, and complex behaviors.

For language models: Supervised = "Copy these good responses." RL = "Try many responses, I will tell you which ones I prefer."

The RL Framework: Five Key Components

Overview

Every RL system has five key pieces. Think of it like a video game. The following table maps each RL concept to its equivalent language model:

RL Term	Video Game Analogy	LLM Translation
Agent	The player	The language model making decisions
Environment	The game world	Everything the model interacts with (the user, the conversation)
State	The current screen	What is happening right now (the prompt + text generated so far)
Action	Controller input	Choosing the next word (token)
Reward	Score / points	Feedback on how helpful the response was

These five pieces connect in a loop that repeats at every step:

Agent in State → Takes Action → Gets Reward → Moves to New State → Repeat

For a language model, this loop runs once for every single word it writes. The model looks at everything so far (state), picks a word (action), gets some signal about how things are going (reward), and the state updates to include the new word. Then it picks the next word and the cycle continues until the response is complete.

The Policy: The Agent's Strategy

The **policy** is the agent's decision-making strategy. It answers: "Given what has happened so far, what should I do next?"

For language models:

- Input: The prompt and everything written so far
- Output: Probability distribution over all possible next words
- Example: Given "The cat sat on the", what word comes next?

The Policy Function

Formal Math	What It Really Means
$\pi(a \mid s)$	Probability of choosing action a when in state s
$\pi_\theta(y_t \mid x, y_{1:t-1})$	Chance of picking word y_t given prompt x and previous words

Symbol Translation:

Symbol	Meaning
π (pi) = Policy	The brain's decision-making process
θ (theta) = Parameters	The brain's knowledge (billions of numbers in the model)
a = Action	The choice to make (which word to write)
s = State	Current situation (everything so far)
y_t = Token at time t	The t-th word in the response
x = Prompt	The user's question/input
$y_{1:t-1}$ = Previous tokens	All words written before word t
$\mid$ = "Given that"	"Knowing…" or "Based on…"

Why condition on previous words with $\mid$? Because language has *context dependence*: "The cat" might be followed by "sat" or "slept," and "The cat sat" might be followed by "on" or "down." Each choice depends on what came before!

The policy in action with real numbers:

State s: "The cat sat on the"

Question: What word should come next?

The policy π_θ outputs the probabilities for each possible word:

Possible Word	**Formal**	**Probability**	**Percentage**
"mat"	$\pi_\theta(\text{mat} \mid s)$	0.35	35%
"floor"	$\pi_\theta(\text{floor} \mid s)$	0.25	25%
"couch"	$\pi_\theta(\text{couch} \mid s)$	0.20	20%
"table"	$\pi_\theta(\text{table} \mid s)$	0.10	10%
"roof"	$\pi_\theta(\text{roof} \mid s)$	0.05	5%
Others	$\pi_\theta(\text{other} \mid s)$	0.05	5%
Total		**1.00**	**100%**

How does the model choose? The model uses these probabilities like a weighted die: most of the time (35%), it picks "mat;" sometimes (25%), it picks "floor;" rarely (5%), it picks "roof." This randomness is actually useful because it helps the model explore different possibilities. (The degree of randomness is controlled by a setting called *temperature*, which we will revisit in Chapter 13 when we discuss how to use compute at inference time.)

What happens during RL training?

Scenario 1: The response with "mat" gets a high reward (+8). Training increases $\pi_\theta(\text{mat} \mid s)$ from 0.35 to 0.45, and response with "mat" becomes more likely!

Scenario 2: The response with "roof" gets a low reward (-2). Training decreases $\pi_\theta(\text{roof} \mid s)$ from 0.05 to 0.02. Response with "roof" becomes less likely!

Training adjusts probabilities based on what leads to good outcomes. Good outcome → increase probability. Bad outcome → decrease probability. Over millions of examples, the model learns an excellent policy!

The Value Function: Looking Ahead

The value function estimates: "How good is my current situation?"

It is like looking at a chess board mid-game and thinking: "I am in a strong position, likely to win!" (high value) or "I am in trouble" (low value).

For language models, halfway through writing a response, the value function estimates whether things are going well ("probably get +8 reward at the end") or poorly ("probably get -3 reward").

The Value Function

Formal Math	What It Really Means
$V^{\pi}(s) = \mathbb{E}_{\pi}\left[\sum_{t=0}^{\infty} \gamma^t r_t \,\middle|\, s_0 = s\right]$	Expected total future reward starting from state s

Breaking down each piece and WHY we use it:

Component	Purpose
$V^{\pi}(s)$	"Value of state s": how good is this situation?
$\mathbb{E}_{\pi}[\cdot]$	Expected value: average over many possible futures (because the future is uncertain)
$\sum_{t=0}^{\infty}$	Sum all future rewards from now to the end (we care about the total, not just the immediate reward)
γ^t	Discount factor raised to power t (immediate rewards matter more than distant ones)
r_t	Reward at time step t (the feedback we get)
$s_0 = s$	Starting from this specific state s

Why the discount factor γ (typically 0.99)? It makes rewards far in the future count slightly less. There is more uncertainty about the distant future, it ensures the series converges to a finite value, and (like money) $100 today is worth more than $100 in 10 years. For language models, we care most about the final response quality.

Computing value with actual numbers

Scenario: You are writing a math solution, currently halfway done.

Current state s: "To solve this, I will first…"

Path A (60% likely): Complete the answer correctly.

- Step 1: Write next sentence → reward $r_1 = 0$ (neutral)

- Step 2: Write another sentence → reward $r_2 = 0$ (neutral)
- Step 3: Finish with correct answer → reward $r_3 = +10$ (great!)
- Total: $0 + 0 + 10 = 10$

Path B (30% likely): Make an error, then correct it.

- Step 1: Make mistake → reward $r_1 = -2$ (penalty)
- Step 2: Realize and backtrack → reward $r_2 = -1$ (small penalty)
- Step 3: Fix and complete → reward $r_3 = +8$ (good!)
- Total: $-2 + (-1) + 8 = 5$

Path C (10% likely): Give up or produce wrong answer.

- Step 1: Incomplete response → reward $r_1 = -5$ (bad!)
- Total: -5

Computing the value $V^\pi(s)$:

$$\begin{aligned} V^\pi(s) &= P(\text{Path A}) \times \text{Reward}_A + P(\text{Path B}) \times \text{Reward}_B + P(\text{Path C}) \times \text{Reward}_C \\ &= 0.6 \times 10 + 0.3 \times 5 + 0.1 \times (-5) \\ &= 6 + 1.5 - 0.5 = 7 \end{aligned}$$

Value = 7. This state is pretty good! On average, following our policy from here will get us +7 reward. High value state means keep doing what we are doing; low value state means try something different.

Adding the discount factor

In the example above, we treated all rewards equally regardless of when they arrive. But in practice, rewards that arrive sooner are worth slightly more than rewards that arrive later. That is what the discount factor captures.

With discount factor $\gamma = 0.99$:

If rewards are spread over time:

$$V^{\pi}(s) = r_0 + \gamma r_1 + \gamma^2 r_2 + \gamma^3 r_3 + \cdots$$
$$= 0 + 0.99 \times 0 + 0.99^2 \times 10 = 0.98 \times 10 = 9.8$$

The discount factor slightly reduces the value of future rewards, but with $\gamma = 0.99$, the effect is small.

The Goal of RL

What Reinforcement Learning Tries To Do

Formal Math	What It Really Means
$\pi^* = \arg\max_\pi \mathbb{E}_{\tau\sim\pi}\left[\sum_{t=0}^{T} r_t\right]$	Find the best strategy π^* that maximizes average total reward

Symbol Guide:

Symbol	Meaning
π^* (pi-star)	The optimal (best possible) policy
$\arg\max_\pi$	"Find the π that maximizes…": searching over all possible strategies
$\mathbb{E}[\cdot]$	Average/expected value over many tries (because of randomness)
$\tau \sim \pi$	A trajectory (complete episode) generated by following policy π
$\sum_{t=0}^{T} r_t$	Total reward collected (sum from start $t = 0$ to end T)

In simple terms with an example:

Goal: Find the decision-making strategy that gets the highest average score.

The search process (simplified):

Strategy 1: Always pick the most likely word. Try 100 prompts. Average reward: +5.2.

Strategy 2: Sometimes explore unusual words. Try 100 prompts. Average reward: +6.8 (better!).

Strategy 3: Carefully balance common and creative words. Try 100 prompts. Average reward: +8.1 (even better!).

Then we keep the best, make variations, test them, and continue to improve. After millions of iterations, we converge to the best strategy π^* and cannot improve further.

Concrete language model example

Prompt: "Explain machine learning"

Bad strategy (low reward):

> *"Machine learning is when computers use algorithms to learn from data and make predictions or decisions without being explicitly programmed…" [continues with jargon]*

Average reward: +3 (too technical, not helpful for beginners).

Good strategy (high reward):

> *"Think of machine learning as teaching a child to recognize animals. You show them many pictures and say 'that is a dog, that is a cat.' Eventually, they learn the patterns and can identify animals that they have never seen before. Computers do something similar with data!"*

Average reward: +9 (clear, accessible, helpful!).

The magic of RL: Through millions of trials, the model automatically discovers that clear explanations lead to high reward, using analogies leads to high reward, too much jargon leads to low reward, and ignoring the user leads to low reward. Nobody explicitly programmed these preferences. The model learned them from feedback!

The Road Ahead

You now have the complete vocabulary of reinforcement learning: agents that follow policies, environments that return rewards, value functions that estimate future outcomes, and an optimization goal that ties it all together. Every technique in the rest of this book is built on these foundations.

In the next chapter, we will set up a free coding environment (Chapter 4) so

that you can start running experiments hands-on. Then, in Chapter 5, we will take our first concrete step toward alignment: supervised fine-tuning, where we teach the model good behavior by showing it curated examples before any RL begins.

As we move deeper into the book, you will see these RL fundamentals appear again and again. In Chapter 6, we will meet PPO, an algorithm that uses the policy, value function, and reward signal together in a carefully choreographed training loop. In Chapter 10, we will encounter GRPO, a newer algorithm that achieves similar results while removing the need for the value function entirely. The concepts you have built here are the foundation for understanding both.

Companion notebook. The code for this chapter is in `code/notebooks/part1_founda` in the book repository at `github.com/arunpshankar/packt-final`. On GitHub, navigate to the file and replace `github.com` with `githubtocolab.com` in the URL to open it directly in Colab.